
/ Blog

Home / News & insights / Research

Does GitHub Copilot improve code quality? Here's what the data says

Findings in our latest study show that the quality of code written with GitHub Copilot is significantly more functional, readable, reliable, maintainable, and concise.



Jared Bauer · @jaredsb

November 18, 2024

Updated February 6, 2025

🕒 8 minutes

Share:



AI has fundamentally changed software development in the two years since [GitHub Copilot](#) was released to the public. In that time, GitHub Copilot has [helped developers](#)

/ Blog

But the question remains: is the quality of code written using GitHub Copilot objectively better or worse?

To answer this, we conducted a randomized controlled trial to understand how **functional, readable, reliable, maintainable, concise, and likely to be approved** code authored using GitHub Copilot is.

In the study, we recruited 202 developers with at least five years of experience. Half were randomly assigned GitHub Copilot access and the other half were instructed not to use any AI tools. The participants were all asked to complete a coding task writing API endpoints for a web server. We then evaluated the code with unit tests and with an expert review conducted by developers.

Our findings overall show that code authored with GitHub Copilot has increased functionality and improved readability, is of better quality, and receives higher approval rates.

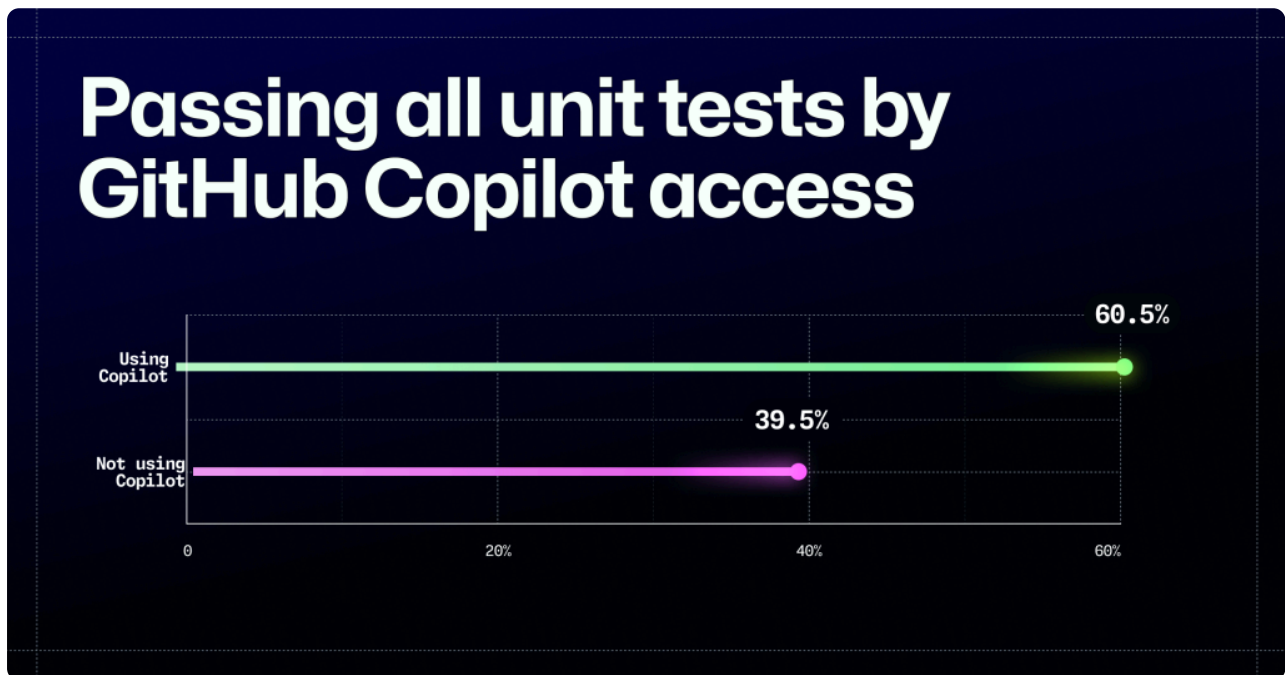
Key findings from this study:

1. **Increased functionality:** developers with GitHub Copilot access had a 53.2% greater likelihood of passing all 10 unit tests in the study, indicating that GitHub Copilot helps developers write more functional code by a wide margin.
2. **Improved readability:** in blind reviews, code written with GitHub Copilot had significantly fewer code readability errors, allowing developers to write 13.6% more lines of code, on average, without encountering readability problems.
3. **Overall better quality code:** readability improved by 3.62%, reliability by 2.94%, maintainability by 2.47%, and conciseness by 4.16%. All numbers were statistically significant. These quality improvements were consistent with those found in the [2024 DORA Report](#).
4. **Higher approval rates:** developers were 5% more likely to approve code written with GitHub Copilot, meaning that such code is ready to be merged sooner, speeding up the time to fix bugs or deploy new features.

Here's a deep dive into what we found:

/ Blog

If code doesn't work, it's impossible to say it's high quality. So, we looked at **functionality** and measured it by analyzing how many unit tests the code passed. We found that code authored using GitHub Copilot passed significantly more tests ($p < 0.01$). **In fact, the developers with GitHub Copilot access had a 53.2%% greater likelihood to pass all 10 unit tests in the study** ($p < 0.01$). This means that using GitHub Copilot helps developers write code that is more functional by a wide margin.



Editor's note: The image above was updated to show just the percentage of study participants who passed all 10 unit tests. An earlier version included both those who did and did not pass all unit tests in the study. We also made an additional change after our dataset for the number of unit tests passed by each submission was updated to remove a submission that was found to be invalid. This impacted the statistics for the number of unit tests passed by Copilot access. The findings remained statistically significant and the values have been updated to reflect the final dataset.

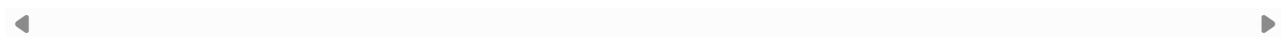
Developers found code written using GitHub Copilot easier to read

The 25 developers who authored code that passed all 10 unit tests from the first phase of the study were randomly assigned to do a blind review of the anonymized submissions,

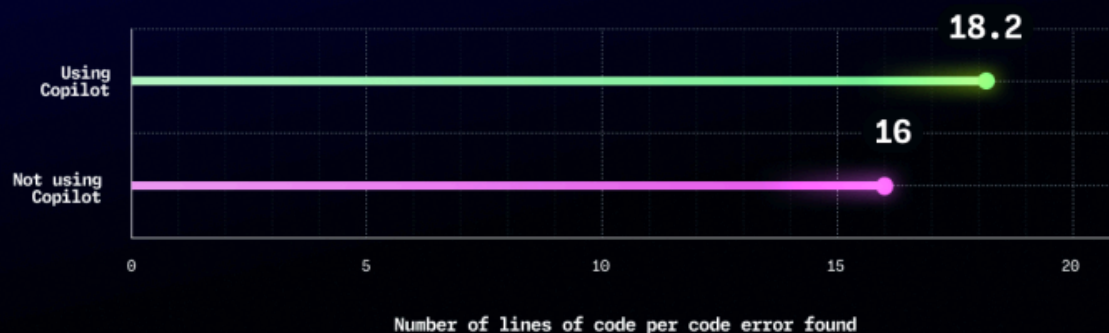
/ Blog

Our analysis of the developers' line-by-line code review showed that code written using GitHub Copilot had significantly fewer code errors: developers using GitHub Copilot wrote 18.2 lines of code per code error, but only 16.0 without. That equals 13.6% more lines of code with GitHub Copilot on average without a code error ($p=0.002$). This can translate into real-time savings, as each of these code errors require action from a developer. For example, without using GitHub Copilot, teams may face up to 13% more comments or suggestions to address, which accumulates over time.

	Mean # of code errors	Mean lines of code	Avg lines of code per code error	% difference
Using GitHub Copilot	4.63	84.3	18.2	13.6%
Not using GitHub Copilot	5.35	85.7	16.0	-11.9%



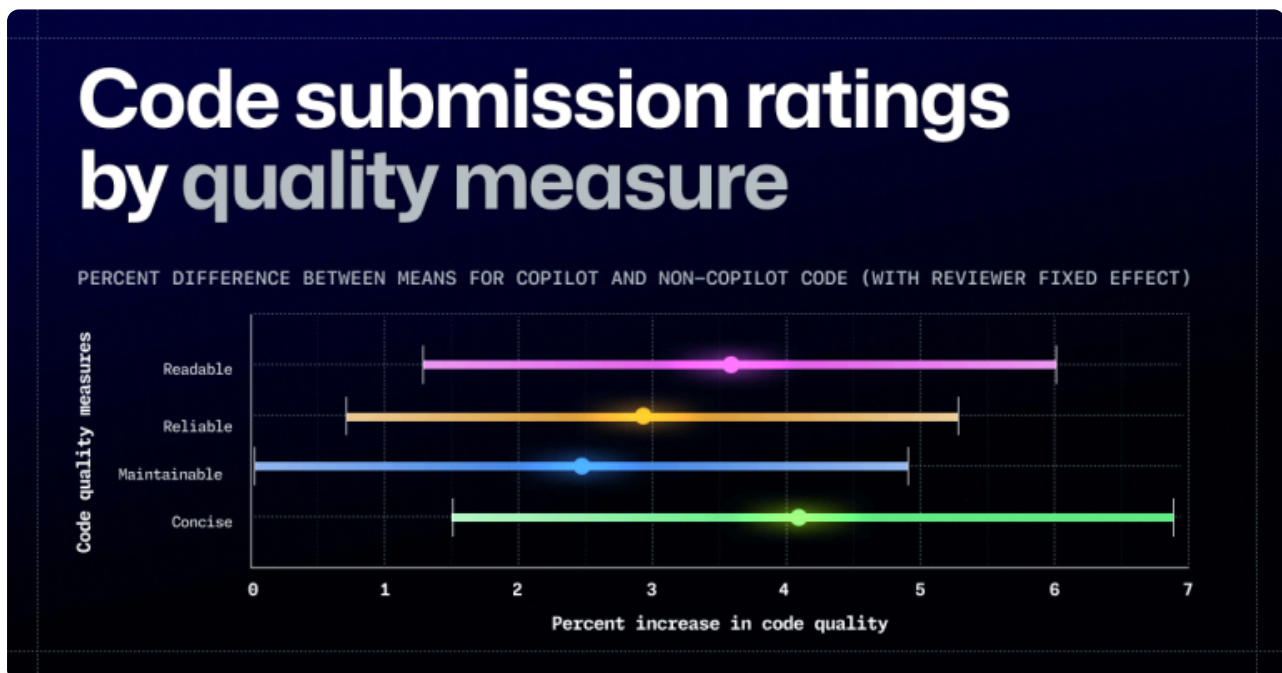
Lines of code authored per code error



/ Blog

respectively). While these differences were small, they were statistically significant and do contribute to a better codebase.

Dependent Variable	Mean difference	P-value
Readable	3.62%	0.003
Reliable	2.94%	0.01
Maintainable	2.47%	0.041
Concise	4.16%	0.002



Code authored using GitHub Copilot was more likely to be approved

Finally, we found that developers were 5% more likely to approve code authored using GitHub Copilot, too ($p=0.014$). In real-world settings, this means that developers using

/ Blog

Percent of code submissions approved when authored with or without GitHub Copilot



The bottom line

So, what do these findings say about how GitHub Copilot improves code quality? While the number of commits and lines of code changed was significantly higher for the GitHub Copilot group, the average commit size was slightly smaller. This suggests that GitHub Copilot enabled developers to iterate on the code to improve its quality. Our hypothesis is that because developers spent less time making their code functional, they were able to focus more on refining its quality. This aligns with our previous findings that developers felt more confident using GitHub Copilot. It also demonstrates that with the greater confidence GitHub Copilot gave them, they were likely empowered to iterate without the fear of causing errors in the code.

As the first controlled study to examine GitHub Copilot's impact on code quality, it shows that GitHub Copilot aids in writing high quality code. We hypothesize that other studies might not have found an improvement in code quality with GitHub Copilot, not because of the tool itself, but because developers may have lacked the opportunity or incentive to focus on quality. This data builds on our previous research indicating that GitHub Copilot is a powerful product that helps developers code quicker and increase job satisfaction, as well as empowers teams to move fast and supercharge their creativity and innovation.

Here, in the GitHub Customer Research team, we're constantly conducting new research on the efficacy of our products as we work to be the home of 1 billion developers—so, stay tuned for more insights and developments in the near future.

/ Blog

In the first phase of the study, we recruited 243 developers with at least five years of Python experience. They were randomly assigned to either use GitHub Copilot or not. Each group completed a coding exercise for a web server of fictional restaurant reviews, with 10 unit tests to assess functionality. We received valid submissions from 202 developers: 104 with GitHub Copilot and 98 without.

In the second phase, developers were randomly assigned submissions to review using a provided rubric. They were blind to whether the code was authored with GitHub Copilot. Each submission was reviewed by at least 10 different participants, resulting in 1,293 reviews. The developers used the rubric to provide a line-by-line review that focused on identifying code errors. They also provided an overall evaluation of the submission for **readability, reliability, maintainability, and conciseness**, and if the submission should be **approved**. For additional questions about the methodology, please reach out to press@github.com

How do we define a code error?

In this study, we defined code errors as any code that reduces the ability for the code to be easily understood. This did not include functional errors that would prevent the code from operating as intended, but instead errors that represent poor coding practices. These code errors were derived from academic literature on code readability¹ and code complexity². The code errors were used in the rubric provided during the code review. They included: inconsistent naming, unclear identifiers, excessive line length, excessive whitespace, missing documentation, repeated code, excessive branching or loop depth, insufficient separation of functionality, and variable complexity.

***Acknowledgments:** we'd like to thank Lizzie Redford, Ph.D. and [Sida Peng, Ph.D.](#) for their support on the study design and statistical analysis of this research.*

Ready to build high-quality code quickly?

[Get started with GitHub Copilot >](#)

Notes

/ Blog

<https://doi.org/10.1145/1390630.1390647> 

2. D. Beyer and A. Fararooy, "A Simple and Effective Measure for Complex Low-Level Dependencies," 2010 IEEE 18th International Conference on Program Comprehension, Braga, Portugal, 2010, pp. 80-83, doi: 10.1109/ICPC.2010.49.
keywords: {Software measurement;Lab-on-a-chip;Size measurement;Length measurement;Application software;Software systems;Frequency;Stability;Guidelines;Program processors;Software Measure;Dependency Analysis;Refactoring;Program Understanding}, 

Tags :

code quality

GitHub Copilot

Written by



Jared Bauer

[@jaredsb](#)

Staff Researcher, GitHub

Related posts

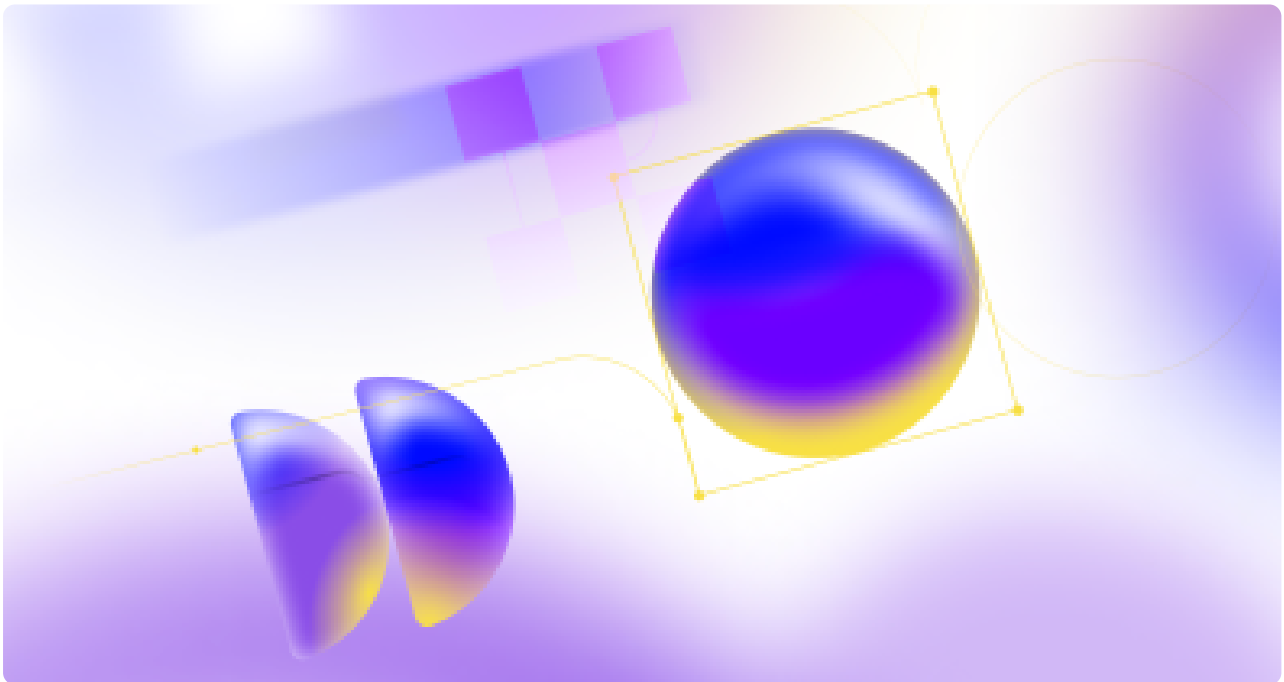
/ Blog



GitHub Availability Report: February 2025

In February, we experienced two incidents that resulted in degraded performance across GitHub services.

Jakub Oleksy



GitHub Availability Report: January 2025

In January, we experienced two incidents that resulted in degraded performance across GitHub services.

[/ Blog](#)

GitHub Copilot: The agent awakens

Introducing agent mode for GitHub Copilot in VS Code, announcing the general availability of Copilot Edits, and providing a first look at our SWE agent.

Thomas Dohmke

Explore more from GitHub



Docs

Everything you need to master GitHub, all in one place.

Go to Docs [↗](#)

/ Blog

Build what's next on GitHub, the place for anyone from anywhere to build anything.

Start building >



Customer stories

Meet the companies and engineering teams that build with GitHub.

Learn more [↗](#)



GitHub Universe 2024

Get tickets to the 10th anniversary of our global developer event on AI, DevEx, and security.

Get tickets [↗](#)

We do newsletters, too

Discover tips, technical guides, and best practices in our biweekly newsletter just for devs.

Your email address

☐ Yes please, I'd like GitHub and affiliates to use my information for personalized communications, targeted advertising and campaign effectiveness. See the [GitHub Privacy Statement](#) for more details.



Product	Platform	Support	Company
Features	Developer API	Docs	About
Security	Partners	Community Forum	Blog
Enterprise	Atom	Training	Careers
Customer Stories	Electron	Status	Press
Pricing	GitHub Desktop	Contact	Shop
Resources			